

Task 1: Install TensorFlow & Keras in a Virtual Environment

Step 1: Create a Virtual Environment

Windows

```
python -m venv tf_env
```

Linux/macOS

```
python3 -m venv tf_env
```

Step 2: Activate the Environment

Windows

```
tf_env\Scripts\activate
```

Linux/macOS

```
source tf_env/bin/activate
```

Step 3: Install TensorFlow

```
pip install tensorflow
```

Step 4: Verify Installation

Start Python:

```
python
```

Run:

```
import tensorflow as tf
```

```
print("TensorFlow Version:", tf.__version__)
```

Expected Output

```
TensorFlow Version: 2.x.x
```

If no error appears, TensorFlow is installed correctly.

Task 2: Keras Sequential Model

```
import tensorflow as tf
```

```
from tensorflow.keras.models import Sequential
```

```

from tensorflow.keras.layers import Dense

# Create Model
model = Sequential([

    Dense(32, activation='relu', input_shape=(10,)),

    Dense(16, activation='relu'),

    Dense(1, activation='sigmoid')

])

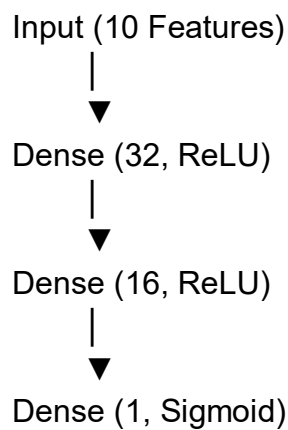
```

```

# Print Summary
model.summary()

```

Model Architecture



Expected Summary

Model: "sequential"

Layer (type)	Output Shape	Param #
Dense	(None,32)	352
Dense	(None,16)	528
Dense	(None,1)	17

Total params: 897

Task 3: Functional API Model

```
import tensorflow as tf
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
```

```
# Input Layer
inputs = Input(shape=(10,))
```

```
# Hidden Layer
x = Dense(20, activation='relu')(inputs)
```

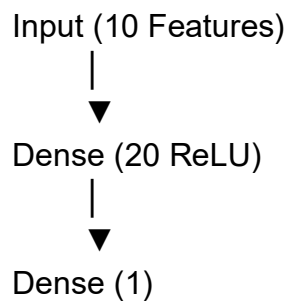
```
# Output Layer
outputs = Dense(1)(x)
```

```
# Functional Model
model = Model(inputs=inputs, outputs=outputs)
```

```
# Summary
model.summary()
```

```
print("Total Parameters:", model.count_params())
```

Architecture



Parameter Calculation

First Dense Layer:

$$(10 \times 20) + 20 = 220$$

Output Layer:

$$(20 \times 1) + 1 = 21$$

Total:

$$220 + 21 = 241$$

Expected output:

Total Parameters: 241

Task 4: Check CPU or GPU

```
import tensorflow as tf

print("TensorFlow Version:", tf.__version__)

print("\nAvailable CPU Devices:")
print(tf.config.list_physical_devices('CPU'))

print("\nAvailable GPU Devices:")
print(tf.config.list_physical_devices('GPU'))

# Deprecated in newer TensorFlow versions but included for compatibility
try:
    print("\nGPU Available:", tf.test.is_gpu_available())
except AttributeError:
    print("\ntf.test.is_gpu_available() is deprecated.")

if tf.config.list_physical_devices('GPU'):
    print("\nTensorFlow is using GPU.")
else:
    print("\nTensorFlow is using CPU.")
```

Possible Output (CPU)

Available CPU Devices:
[PhysicalDevice(name='/physical_device:CPU:0')]

Available GPU Devices:
[]

TensorFlow is using CPU.

Possible Output (GPU)

Available CPU Devices:
[PhysicalDevice(name='/physical_device:CPU:0')]

Available GPU Devices:
[PhysicalDevice(name='/physical_device:GPU:0')]

TensorFlow is using GPU.

Note: In TensorFlow 2.10+ and newer, `tf.test.is_gpu_available()` is deprecated. The recommended way is `tf.config.list_physical_devices('GPU')`.

Task 5: Functional API Model with Two Inputs

Example: Predict whether a Spotify user is likely to buy a Premium subscription based on **Age** and **Number of Songs Listened**.

```
import tensorflow as tf
from tensorflow.keras.layers import Input, Dense, Concatenate
from tensorflow.keras.models import Model

# Input 1: Age
age_input = Input(shape=(1,), name="Age")

# Input 2: Songs Listened
songs_input = Input(shape=(1,), name="Songs")

# Process Each Input
age_branch = Dense(8, activation="relu")(age_input)

songs_branch = Dense(8, activation="relu")(songs_input)

# Merge Inputs
merged = Concatenate()([age_branch, songs_branch])

# Hidden Layer
hidden = Dense(16, activation="relu")(merged)

# Output Layer
output = Dense(1, activation="sigmoid")(hidden)

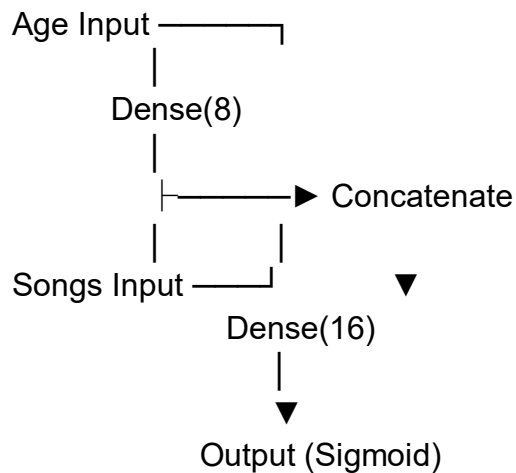
# Create Model
model = Model(
    inputs=[age_input, songs_input],
    outputs=output
)

# Compile Model
model.compile(
    optimizer="adam",
```

```
    loss="binary_crossentropy",  
    metrics=["accuracy"]  
)
```

```
# Print Summary  
model.summary()
```

Architecture



Possible Changes or Errors

When running this code, you may need to:

- Ensure TensorFlow 2.x is installed.
- Provide **two separate input arrays** during training, for example:

```
model.fit([age_data, songs_data], labels, epochs=5)
```

- If you use a single input array instead of a list of two arrays, Keras will raise an input shape mismatch error.